

Chapter 33 - IE Script

IE Script is the second of Intuition Engine's two debugging surfaces. Where the Machine Monitor (Chapter 32) is interactive, typed one command at a time, IE Script is a small scripting language that runs from a file: it can drive long sequences of actions, watch for events, react to faults, capture frames, and script regression tests of your own programs.

A script file has the extension `.ies`. You launch one from BASIC with `RUN "path/to/file.ies"`, or hand it to the system at start.

33.1 The shape of a script

An IE Script file is a sequence of statements. Expressions are familiar: integers, booleans, strings (with 'single' or "double" quotes), arithmetic, comparison, and/or/not, if/then/elseif/else/end, while, for, function definitions, and tables.

Comments begin with `--`.

```
-- print a banner and pause one frame
print("hello from script")
sys.sleep_frames(1)

-- toggle the cursor for one second
for i = 1, 60 do
    term.cursor(i % 2 == 0)
    sys.sleep_frames(1)
end
```

33.2 Global tables

The script's environment exposes Intuition Engine's subsystems as a small set of global tables. Each table is a namespace of functions; the descriptions below name each function by its full path.

33.2.1 `sys`

Coarse-grained system control:

Function	Purpose
<code>sys.sleep_frames(n)</code>	Yield until <code>n</code> video frames have elapsed
<code>sys.sleep_ms(ms)</code>	Yield until <code>ms</code> milliseconds elapse
<code>sys.frames()</code>	Number of frames since boot
<code>sys.now_ms()</code>	Master clock in milliseconds
<code>sys.reset()</code>	Hard reset Intuition Engine
<code>sys.run(path)</code>	Load a program image and run it
<code>sys.quit()</code>	Exit the script (back to BASIC or to caller)
<code>sys.exit(code)</code>	Exit Intuition Engine with status code

33.2.2 cpu

Active-CPU control. The active CPU is whatever the program executor most recently switched to, or the CPU you nominate with `cpu.switch`.

Function	Purpose
<code>cpu.switch(name)</code>	Switch to a named CPU ("ie64", etc.)
<code>cpu.name()</code>	Name of the active CPU
<code>cpu.pc()</code> / <code>cpu.set_pc(v)</code>	Get/set the PC
<code>cpu.reg(name)</code> / <code>cpu.set_reg(name, v)</code>	Get/set a register
<code>cpu.step()</code>	Single-step
<code>cpu.freeze()</code> / <code>cpu.thaw()</code>	Stop / resume the active CPU
<code>cpu.running()</code>	True if the active CPU is running

33.2.3 mem

Bus accesses. Width is given by the function name.

Function	Purpose
<code>mem.peek8(addr)</code>	Read one byte
<code>mem.peek16(addr)</code>	Read one little-endian word
<code>mem.peek32(addr)</code>	Read one little-endian long
<code>mem.poke8(addr, v)</code>	Write one byte
<code>mem.poke16(addr, v)</code>	Write one word
<code>mem.poke32(addr, v)</code>	Write one long
<code>mem.read(addr, n)</code>	Read <i>n</i> bytes, return as a string
<code>mem.write(addr, s)</code>	Write the bytes of string <i>s</i> to <i>addr</i>
<code>mem.fill(addr, n, byte)</code>	Fill <i>n</i> bytes with one byte

33.2.4 term

Terminal control.

Function	Purpose
<code>term.print(s)</code>	Print a string to the terminal
<code>term.clear()</code>	Clear the terminal
<code>term.cursor(on)</code>	Show or hide the cursor
<code>term.echo(on)</code>	Enable / disable local echo
<code>term.input()</code>	Return a line of input, or <code>nil</code> if none yet

33.2.5 audio

Audio mixer + per-engine control.

Function	Purpose
<code>audio.volume(v)</code>	Set the master volume (0–1)
<code>audio.mute(on)</code>	Mute / unmute the mixer
<code>audio.freeze()</code> / <code>audio.thaw()</code>	Freeze / thaw the audio subsystem
<code>audio.engine(name).play(file)</code>	Play file on the named engine
<code>audio.engine(name).stop()</code>	Stop the engine

The engine names match the BASIC verbs in Chapter 11: "sid", "psg", "sn", "pokey", "ted", "ahx", "mod", "wav", "paula", "sfx".

33.2.6 video

Video subsystem.

Function	Purpose
<code>video.mode()</code> / <code>video.set_mode(m)</code>	Read / set VideoChip resolution
<code>video.fb_base()</code>	Current framebuffer base address
<code>video.frame()</code>	Index of the most recent rendered frame
<code>video.snapshot(file)</code>	Save the current frame as a PNG

33.2.7 rec

Video recording.

Function	Purpose
<code>rec.start(file)</code>	Begin recording to file
<code>rec.stop()</code>	Stop and finalise
<code>rec.is_recording()</code>	Return <code>true</code> while recording

33.2.8 dbg

The debugging surface. This is the bridge between scripts and the Machine Monitor.

Function	Purpose
<code>dbg.cmd(line)</code>	Run a monitor command, return its text output
<code>dbg.break_at(addr)</code>	Set a code breakpoint
<code>dbg.watch(addr, size, mode)</code>	Set a memory watchpoint
<code>dbg.on_fault(cb)</code>	Register a callback for trap/fault events
<code>dbg.wait_fault(timeout_ms)</code>	Block until a fault fires
<code>dbg.disasm(addr, count)</code>	Disassemble <code>count</code> instructions

Function	Purpose
<code>dbg.regs()</code>	Return a table of register name → value
<code>dbg.history(count)</code>	Return recent execution history

`dbg.cmd("d 1000 20")` is the script equivalent of typing `d 1000 20` at the monitor prompt. The full command surface of Chapter 32 is available through `dbg.cmd`.

33.2.9 sym and regions

Symbol lookup and memory-region maps:

Function	Purpose
<code>sym.lookup(name)</code>	Return the address of a symbol
<code>sym.name(addr)</code>	Return the symbol at <code>addr</code> if any
<code>regions.list()</code>	Table of {base, end, name} for each MMIO region

33.2.10 bit32

Bitwise utilities (the language does not have native bit operators): `bit32.band`, `bor`, `bxor`, `bnot`, `lshift`, `rshift`, `arshift`, `lrotate`, `rrotate`, `btest`, `extract`, `replace`.

33.3 Event-driven scripting

Scripts can be linear (do this, then that, then exit) or event-driven (wait for things to happen, react). The two main event sources are video frames and faults.

```
-- count frames until something interesting happens
local stop = false
dbg.on_fault(function(ev)
    print(string.format("fault at PC=%08x cause=%d", ev.pc, ev.cause))
    stop = true
end)

while not stop do
    sys.sleep_frames(1)
end
```

The callback runs synchronously when the CPU faults; the main script body resumes on the next frame.

33.4 Running scripts

There are three ways to start a script:

1. **From BASIC:** RUN "path/to/file.ies" from the BASIC prompt loads and starts the script. BASIC waits until the script returns or calls `sys.quit`.
2. **From the monitor:** at the `(cpu)>` prompt, type `load_script path/to/file.ies` to run a script while keeping the monitor open. Faults that fire during the script return control to the monitor.
3. **At launch:** the boot environment can be configured to run a script as the first user-facing action, useful for automated regression suites.

A script that fails - syntax error, uncaught error, or `explicit error(msg)` - prints the message to the terminal and returns control to whichever surface launched it.

33.5 A small example

A script that runs a fresh program, waits ten frames, captures a PNG, and exits:

```
sys.reset()
sys.run("demo.ie64")
sys.sleep_frames(10)
video.snapshot("frame10.png")
sys.quit()
```

A more substantial example that sets a breakpoint, runs a known program, dumps registers at the break, and continues to exit:

```
sys.run("test.ie64")
dbg.break_at(0x2000)

local hit = false
dbg.on_fault(function(ev)
  if ev.kind == "breakpoint" then
    local r = dbg.regs()
    print("at $2000:")
    for k, v in pairs(r) do
      print(string.format("  %-4s = %08x", k, v))
    end
    hit = true
  end
end)

while not hit do
  sys.sleep_frames(1)
end
```

33.6 What comes next

Part IV ends here. Part V picks up the I/O side of Intuition Engine: disk and file I/O (Chapter 34), the `H0ST` command and its appliance-control bridge (Chapter 35), the keyboard / mouse / controller MMIO (Chapter 36), and the serial interface (Chapter 37).